CCE – Problem Statement 60

AVL Deletion of a Node with Two Children

1. Problem Statement

Implement a C++ program to delete a node having two children from an AVL tree using its inorder successor or inorder predecessor and rebalance the tree to maintain the AVL property.

2. Algorithm

1. Start.

2. Initialize an empty AVL tree.

3. Read the number of nodes, n.

4. Read the node values one by one and insert them into the AVL tree. Update heights and rebalance after each insertion.

5. Display the inorder traversal before deletion.

6. Read the key to delete.

7. Ask the user to choose the replacement method: 1 for inorder successor or 2 for inorder predecessor.

8. Search for the key in the AVL tree.

9. If the key is not found, leave the tree unchanged.

10. If the node has zero or one child, replace it with its child (or NULL) and delete the node.

11. If the node has two children:

a. For successor-based deletion, find the smallest node in the right subtree.

b. For predecessor-based deletion, find the largest node in the left subtree.

c. Copy the replacement key into the node to be deleted.

d. Delete the replacement node from its original position.

12. Update heights on the way back through the recursive calls.

13. Rebalance affected nodes using AVL rotations if a balance factor is outside the range -1 to +1.

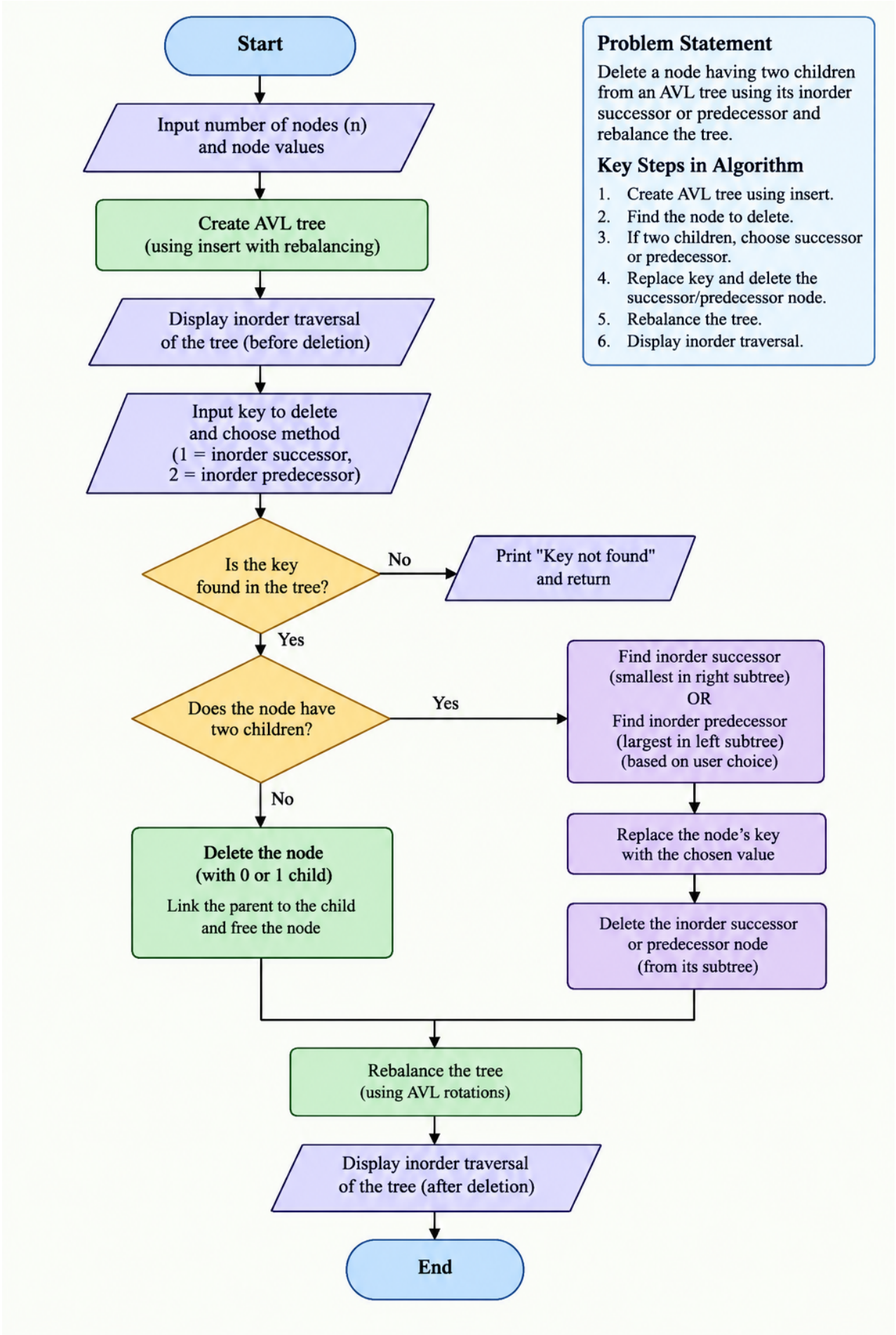
14. Display the inorder traversal after deletion.

15. Free the remaining tree nodes.

16. Stop.

3. Flowchart

The flowchart illustrates the deletion process, replacement choice, and AVL rebalancing.



4. Sample Input

```
7
30 20 40 10 25 35 50
30
1
```

Input explanation: 7 is the number of nodes; the next line contains the node values; 30 is the key to delete; 1 selects the inorder successor. Enter 2 for the inorder predecessor.

5. Expected Output (main traversal lines)

```
AVL tree before deletion:
Inorder traversal: 10 20 25 30 35 40 50

AVL tree after deletion:
Inorder traversal: 10 20 25 35 40 50
```

6. Result

The program deletes the selected node, uses the chosen inorder successor or predecessor when the node has two children, and rebalances the AVL tree to maintain its height-balance property. The inorder traversal displays the remaining keys in sorted order.